MicroserviceBase

v. 0.1.1

Nguyen Huynh Tri Cuong

02.02.2023

Contents

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceBase**.

The sources of **MicroserviceBase** are available in GitHub.

Informations about how to install the **MicroserviceBase** can be found in the README.

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

eventbus_config.py

3.1 Class: EventBusConfig

Imported by:

```
from MicroserviceBase.adapters.config.eventbus_config import EventBusConfig
```

Configuration for EventBusClient-based adapters.

Wraps a JSONP config file path used by EventBusClient.from_config_sync().

3.1.1 Method: load_config

Load and return the parsed config as a dict.

Requires EventBusClient to be installed.

Returns:

/ *Type*: dict /

Parsed configuration dictionary.

3.1.2 Method: to_config_source

Load config and return as a dict with optional field overrides.

Useful for creating derivative clients (e.g., registry or fanout) that share connection settings but differ in exchange configuration.

Arguments:

- `overrides`

  / *Condition*: optional / *Type*: keyword arguments /

  Fields to override in the loaded config (e.g., exchange_name, exchange_handler).

Returns:

/ *Type*: dict /

Config dictionary with overrides applied.

Chapter 4

rabbitmq_config.py

4.1 Class: RabbitMQConfig

Imported by:

```
from MicroserviceBase.adapters.config.rabbitmq_config import RabbitMQConfig
```

Configuration for connecting to RabbitMQ.

4.1.1 Method: to_connection_params

Convert to pika ConnectionParameters kwargs.

Returns:

dict suitable for pika.ConnectionParameters(**kwargs).

4.1.2 Method: from_cmd_args

Parse RabbitMQ config from command-line arguments and environment variables.

Arguments:

- `cmd_args`
  / *Condition*: optional / *Type*: list /
  List of CLI arguments, or None for sys.argv.
- `service_name`
  / *Condition*: optional / *Type*: str /
  Name of the service (used in help text).

Returns:

RabbitMQConfig instance.

Chapter 5

amqp_registry_adapter.py

5.1 Class: AMQPRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.amqp_registry_adapter import
↪  AMQPRegistryAdapter
```

AMQP implementation of the ServiceRegistryPort interface.
Uses RabbitMQ topic exchange for service registration events and fanout exchange for realtime update broadcasts.

5.1.1 Method: register

Register a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

5.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

5.1.3 Method: subscribe_to_events

Subscribe to service registration events.
Runs in the calling thread (blocking). Typically called from a daemon thread.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

5.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

5.1.5 Method: get_update_channel_name

Return the realtime update exchange name.

5.1.6 Method: cleanup_update_exchange

Delete the realtime update exchange (for cleanup on shutdown).

5.1.7 Method: cleanup

Close event subscription resources and clean up the update exchange.

Chapter 6

eventbus_registry_adapter.py

6.1 Class: EventBusRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.eventbus_registry_adapter import
↪  EventBusRegistryAdapter
```

EventBusClient implementation of the ServiceRegistryPort interface.

Uses EventBusClient with TopicExchangeHandler for service registration events and a FanoutExchangeHandler for realtime update broadcasts.

Both clients are created from the same JSONP config with exchange overrides via `EventBusConfig.to_config_source()`.

6.1.1 Method: register

Register a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

6.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

6.1.3 Method: subscribe_to_events

Subscribe to service registration events.

Blocks the calling thread. Typically called from a daemon thread. The handler receives the pika-style callback signature (ch, method, properties, body) for backward compatibility.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

6.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

6.1.5 Method: get_update_channel_name

Return the realtime update exchange name.

6.1.6 Method: cleanup

Close all EventBusClient connections.

Chapter 7

eventbus_adapter.py

7.1 Class: _ChannelProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _ChannelProxy
```

Lightweight proxy that mimics a pika channel for the domain handler.
Translates basic_publish and basic_ack calls into EventBusClient operations.

7.1.1 Method: basic_publish

Publish a reply message via the EventBusClient.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (ignored, EventBusClient uses its configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the reply (the reply_to value).
- `properties`
  / *Condition*: optional / *Type*: object /
  Properties object with correlation_id attribute.
- `body`
  / *Condition*: optional / *Type*: str or bytes /
  The message body (JSON string or bytes).

7.1.2 Method: basic_ack

Acknowledge a message (no-op for EventBusClient which auto-acks).

7.2 Class: _MethodProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _MethodProxy
```

Lightweight proxy that mimics a pika method frame.

7.3 Class: _PropertiesProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _PropertiesProxy
```

Lightweight proxy that mimics pika BasicProperties.

7.4 Class: EventBusTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import
↪  EventBusTransportAdapter
```

EventBusClient implementation of the TransportPort interface.

Creates the underlying EventBusClient from a JSONP configuration file via `EventBusClient.from_config_sync()`.

7.4.1 Method: connect

Create the EventBusClient from the JSONP config and establish connection.

7.4.2 Method: disconnect

Close the EventBusClient connection.

7.4.3 Method: consume

Start consuming RPC requests for a service.

Subscribes to the routing key and blocks the calling thread. Incoming messages are adapted to the pika-style handler signature.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the service identifier.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for message binding.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match the configured exchange).
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

7.4.4 Method: rpc_call

Send an RPC request and wait for the response.

Creates a temporary subscription for the reply, sends the request with reply_to and correlation_id headers, and waits for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict.

7.4.5 Method: publish

Publish a message to the exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str, bytes, or dict /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties (used for headers).

Chapter 8

rabbitmq_adapter.py

8.1 Class: RabbitMQTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.rabbitmq_adapter import
↪  RabbitMQTransportAdapter
```

RabbitMQ implementation of the TransportPort interface.

8.1.1 Method: connect

Establish connection to RabbitMQ.

8.1.2 Method: disconnect

Close connection to RabbitMQ.

8.1.3 Method: connection

Access the underlying pika connection (for backward compat).

8.1.4 Method: consume

Start consuming RPC requests for a service.

Sets up the exchange, queue, binding, and starts blocking consumption.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the queue name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for queue binding.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

8.1.5 Method: rpc_call

Send an RPC request and wait for the response.

Creates a new connection for the RPC call (matching original behavior).

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to serialize as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.
- `timeout`
  / *Condition*: optional / *Type*: int / *Default*: 30 /
  Timeout in seconds for waiting for the response.

Returns:

Parsed response dict.

8.1.6 Method: publish

Publish a message to an exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body (str or bytes).
- `properties`
  / *Condition*: optional / *Type*: pika.BasicProperties /
  Optional pika.BasicProperties.

Chapter 9

fastapi_bridge.py

9.1 Class: FastAPIBridge

Imported by:

```
from MicroserviceBase.adapters.ui_bridge.fastapi_bridge import FastAPIBridge
```

FastAPI implementation of UIBridgePort.

Exposes a REST API that any UI client (Electron, browser, mobile) can call. Also provides a WebSocket endpoint for push-based service updates.

Endpoints: POST /api/request - Route a service request through the transport GET /api/services - Get current services info WS /ws/updates - Real-time service update stream

Requires `fastapi` and `uvicorn` (optional dependencies).

9.1.1 Method: start

Start the FastAPI server in a background thread.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable(request_data, exchange, routing_key) -> response dict.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable() -> services info dict.

9.1.2 Method: stop

Stop the FastAPI server.

9.1.3 Method: broadcast_update

Push a services update to all connected WebSocket clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

Chapter 10

exceptions.py

10.1 Class: ServiceError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceError
```

Base exception for service-related errors.

10.2 Class: TransportError

Imported by:

```
from MicroserviceBase.domain.exceptions import TransportError
```

Raised when a transport-level operation fails.

10.3 Class: ServiceNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceNotFoundError
```

Raised when a requested service is not found in the registry.

10.4 Class: MethodNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import MethodNotFoundError
```

Raised when a requested method is not found on a service.

Chapter 11

messages.py

11.1 Class: ResultType

Imported by:

```
from MicroserviceBase.domain.messages import ResultType
```

Result types for service responses.

11.2 Class: ServiceRequest

Imported by:

```
from MicroserviceBase.domain.messages import ServiceRequest
```

Structured request to a service method.

11.2.1 Method: to_dict

Converts this `ServiceRequest` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this request.

11.2.2 Method: to_json

Converts this `ServiceRequest` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this request.

11.2.3 Method: from_dict

Creates a `ServiceRequest` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the dictionary.

11.2.4 Method: from_json

Creates a `ServiceRequest` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the JSON string.

11.3 Class: ServiceResponse

Imported by:

```
from MicroserviceBase.domain.messages import ServiceResponse
```

Structured response from a service method.

11.3.1 Method: to_dict

Converts this `ServiceResponse` instance to an ordered dictionary.

Returns:

/ *Type*: OrderedDict /
Sorted ordered dictionary representation of this response.

11.3.2 Method: to_json

Converts this `ServiceResponse` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this response.

11.3.3 Method: get_json

Backward-compatible alias for `to_json()`.

Returns:

/ *Type*: str /
JSON string representation of this response.

11.3.4 Method: get_data

Backward-compatible alias for `result_data`.

Returns:

/ *Type*: Any /
The result data of this response.

11.3.5 Method: from_dict

Creates a `ServiceResponse` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the dictionary.

11.3.6 Method: from_json

Creates a `ServiceResponse` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the JSON string.

11.4 Class: ServiceEvent

Imported by:

```
from MicroserviceBase.domain.messages import ServiceEvent
```

Event emitted when service state changes.

11.4.1 Method: to_dict

Converts this `ServiceEvent` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this event.

11.4.2 Method: to_json

Converts this `ServiceEvent` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this event.

11.4.3 Method: from_dict

Creates a `ServiceEvent` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the dictionary.

11.4.4 Method: from_json

Creates a `ServiceEvent` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the JSON string.

Chapter 12

models.py

12.1 Class: MethodArgument

Imported by:

```
from MicroserviceBase.domain.models import MethodArgument
```

Describes a single argument of a service API method.

12.1.1 Method: to_dict

Converts this `MethodArgument` instance to a dictionary.
Returns:

/ *Type*: dict /
Dictionary representation of this argument.

12.1.2 Method: from_dict

Creates a `MethodArgument` instance from a dictionary.
Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing argument fields.

Returns:

/ *Type*: MethodArgument /
New instance populated from the dictionary.

12.2 Class: ServiceMethod

Imported by:

```
from MicroserviceBase.domain.models import ServiceMethod
```

Describes a single API method exposed by a service.

12.2.1 Method: to_dict

Converts this `ServiceMethod` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this method.

12.2.2 Method: from_dict

Creates a `ServiceMethod` instance from a name and dictionary.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  The method name.
- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing method fields.

Returns:

/ *Type*: ServiceMethod /
New instance populated from the dictionary.

12.3 Class: ServiceInfo

Imported by:

```
from MicroserviceBase.domain.models import ServiceInfo
```

Describes a service and its metadata.

12.3.1 Method: to_dict

Converts this `ServiceInfo` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this service info.

12.3.2 Method: from_dict

Creates a `ServiceInfo` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing service info fields.

Returns:

/ *Type*: ServiceInfo /
New instance populated from the dictionary.

Chapter 13

service_base.py

13.1 Class: ServiceBase

Imported by:

```
from MicroserviceBase.domain.service_base import ServiceBase
```

Pure domain base class for services.

All methods used to export APIs of a service must begin with the prefix 'svc_api'. This class handles API discovery, docstring parsing, and request dispatch. Transport and registry interactions are delegated to injected ports.

13.1.1 Method: get_service_info

Return the ServiceInfo dataclass for this service.

13.1.2 Method: get_service_info_dict

Return the service info as a plain dict (backward compat).

13.1.3 Method: serve

Start serving requests via the transport port.

13.1.4 Method: register_service

Register this service via the registry port.

13.1.5 Method: unregister_service

Unregister this service via the registry port.

13.1.6 Method: request_service

Send an RPC request to another service via the transport port.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys (backward compat format).

- `exchange_name`
  / *Condition*: required / *Type*: str /
  The exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  The routing key for the target service.

Returns:

/ *Type*: dict /
The response dict from the target service.

13.1.7 Method: close

Close the service.

13.1.8 Method: create_request_data

Create request data for a given method name and arguments.

13.1.9 Method: get_svc_api_methods_dict

Retrieve all service API methods (methods starting with 'svc_api').

13.1.10 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from their docstrings.

13.1.11 Method: parse_docstring

Parse function's docstring to get arguments and return information.

13.1.12 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

13.1.13 Method: svc_api_get_gui_files

Compress and return GUI files as bytes.

13.1.14 Method: is_specific_request

Check if the request is a specific request. Override in subclasses.

13.1.15 Method: on_specific_request

Handle a specific request. Override in subclasses.

Arguments:

- `request`
  / *Condition*: required / *Type*: str /
  The request method name.
- `body`
  / *Condition*: required / *Type*: dict /
  The parsed request body dict.

Returns:

/ *Type*: ServiceResponse /
A ServiceResponse, or None if not handled.

13.1.16 Method: dispatch_request

Dispatch a parsed request body and return a ServiceResponse.

This is the core domain logic: given a request dict with 'method' and 'args', find the matching API method, call it, and return a structured response.

Arguments:

- `request_body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse with result type and data.

13.1.17 Method: on_request

Handle an incoming request (transport callback signature).

This method is called by the transport adapter. It delegates to dispatch_request for the actual domain logic, then uses the transport to send the response back.

Arguments:

- `ch`
  / *Condition*: required / *Type*: object /
  Channel object from transport.
- `method`
  / *Condition*: required / *Type*: object /
  Delivery method from transport.
- `props`
  / *Condition*: required / *Type*: object /
  Message properties from transport.
- `body`
  / *Condition*: required / *Type*: bytes /
  Raw message body (bytes or dict).

Chapter 14

service_registry.py

14.1 Class: ServiceRegistry

Imported by:

```
from MicroserviceBase.domain.service_registry import ServiceRegistry
```

Pure domain registry that tracks services, handles aliases, and routes requests.

14.1.1 Method: handle_update

Handle a service registration/unregistration event.

Arguments:

- `service_information`
  / *Condition*: required / *Type*: dict /
  Dict with 'info' and 'state' keys.

14.1.2 Method: notify_updates

Notify connected clients about service updates via the registry port.

14.1.3 Method: svc_api_get_services_info

Retrieve information of all services connected to the broker.

Returns:

/ *Type*: dict /
A dictionary containing information of all connected services.

14.1.4 Method: svc_api_get_realtime_update_exchange

Retrieve the exchange name of the realtime update exchange.

Returns:

/ *Type*: str /
The name of the realtime update exchange.

14.1.5 Method: svc_api_update_alias_conf

Update the alias configuration information.

Arguments:

- `alias_string`
  / *Condition*: required / *Type*: str /
  The alias configuration string to be updated.

Returns:

(*no returns*)

14.1.6 Method: svc_api_get_alias_conf

Retrieve the alias configuration string in JSON format.

Returns:

/ *Type*: str /
The alias configuration string in JSON format.

14.1.7 Method: is_specific_request

Check if the request is an alias-routed request.

14.1.8 Method: handle_alias_request

Handle an alias request by routing to the actual service.

Arguments:

- `body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse or raw response dict from the target service.

Chapter 15

factory.py

15.1 Function: create_transport

Create a TransportPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').
- `service_name`
  / *Condition*: optional / *Type*: str /
  Service name for help text (used by 'rabbitmq').
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: TransportPort /
A connected TransportPort instance.

15.2 Function: create_registry

Create a ServiceRegistryPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').

- `service_name`
  / *Condition*: optional / *Type*: str /
  Service name for help text (used by 'rabbitmq').
- `update_exchange_name`
  / *Condition*: optional / *Type*: str /
  Name for the realtime update fanout exchange.
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: ServiceRegistryPort /
A ServiceRegistryPort instance.

15.3 Function: create_ui_bridge

Create a UIBridgePort implementation.

Arguments:

- `bridge_type`
  / *Condition*: optional / *Type*: str /
  Bridge backend identifier. Currently supports 'fastapi'.
- `host`
  / *Condition*: optional / *Type*: str /
  Host to bind to.
- `port`
  / *Condition*: optional / *Type*: int /
  Port to bind to.

Returns:

/ *Type*: UIBridgePort /
A UIBridgePort instance (not yet started).

Chapter 16

registry.py

16.1 Class: ServiceRegistryPort

Imported by:

```
from MicroserviceBase.ports.registry import ServiceRegistryPort
```

Abstract interface for service registration and discovery.
Implementations handle how services announce themselves and how the registry discovers/tracks them.

16.1.1 Method: register

Register a service with the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

16.1.2 Method: unregister

Unregister a service from the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

16.1.3 Method: subscribe_to_events

Subscribe to service registration/unregistration events.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body) for events.

16.1.4 Method: notify_update

Broadcast a services update to all subscribers.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

16.1.5 Method: get_update_channel_name

Return the name of the realtime update channel/exchange.

Returns:

Channel/exchange name as a string.

Chapter 17

transport.py

17.1 Class: TransportPort

Imported by:

```
from MicroserviceBase.ports.transport import TransportPort
```

Abstract interface for message transport.

Implementations provide the actual connectivity (e.g., RabbitMQ, Redis, MQTT). The domain layer depends only on this interface.

17.1.1 Method: connect

Establish connection to the message broker.

17.1.2 Method: disconnect

Close connection to the message broker.

17.1.3 Method: consume

Start consuming messages for a service.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name of the service (used as queue name).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key to bind the queue.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body) for incoming messages.

17.1.4 Method: rpc_call

Send an RPC request and wait for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict to serialize and send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict from the target service.

17.1.5 Method: publish

Publish a message to an exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties dict.

Chapter 18

ui_bridge.py

18.1 Class: UIBridgePort

Imported by:

```
from MicroserviceBase.ports.ui_bridge import UIBridgePort
```

Abstract interface for UI communication bridge.

Implementations provide the actual UI connectivity (e.g., FastAPI REST, gRPC). Any frontend (Electron, browser, mobile) can connect through this port.

18.1.1 Method: start

Start the UI bridge server.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable that accepts a ServiceRequest dict and returns a ServiceResponse dict. Used to route UI requests to services.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable that returns the current services info dict.

18.1.2 Method: stop

Stop the UI bridge server.

18.1.3 Method: broadcast_update

Push a services update to all connected UI clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

Chapter 19

Appendix

About this package:

Table 19.1: Package setup

Setup parameter	Value
Name	MicroserviceBase
Version	0.1.1
Date	02.02.2023
Description	Microservice base framework with pluggable transport and registry adapters
Package URL	python-microservice-base
Author	Nguyen Huynh Tri Cuong
Email	Cuong.NguyenHuynhTri@vn.bosch.com
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.10
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 20

History

0.1.0	02/2023
Initial version	

MicroserviceBase.pdf
Created at 06.02.2026 - 23:47:15
by GenPackageDoc v. 0.16.0